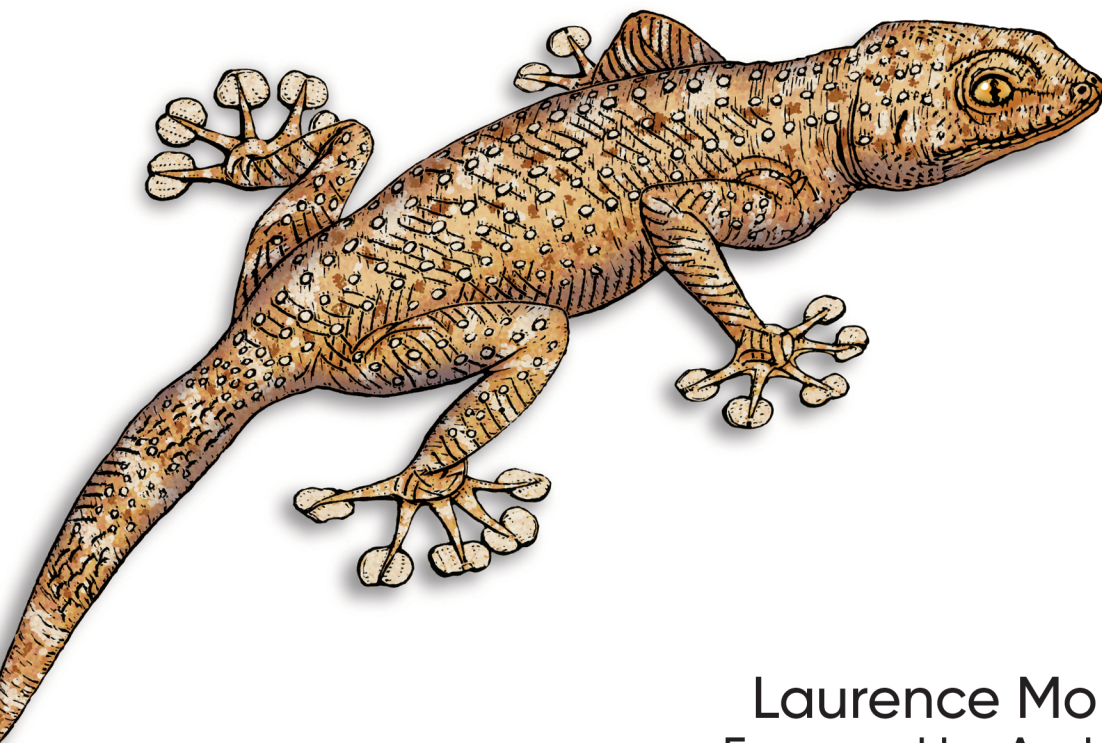


O'REILLY®

AI and Machine Learning for Coders

A Programmer's Guide to Artificial Intelligence



Laurence Moroney
Foreword by Andrew Ng

AI and Machine Learning for Coders

If you're looking to make a career move from programmer to AI specialist, this is the ideal place to start. Based on Laurence Moroney's extremely successful AI courses, this introductory book provides a hands-on, code-first approach to help you build confidence while you learn key topics. All you need is experience with Python and its notation for data and array processing.

You'll learn how to implement the most common scenarios in machine learning, including computer vision, natural language processing (NLP), and sequence modeling for web, mobile, cloud, and embedded runtimes. Most books on machine learning begin with a daunting amount of advanced math. This guide provides practical lessons that let you work directly with the code.

- Understand machine learning basics by working with code samples
- Use TensorFlow to build models for a variety of scenarios
- Build a model with a neural network containing only one neuron
- Implement computer vision, including feature detection in images
- Tokenize and sequence words and sentences with NLP
- Embed your models in Android and iOS devices
- Serve models over the web and in the cloud with TensorFlow Serving

Laurence Moroney leads AI advocacy at Google, teaching software developers how to build AI systems with machine learning. He's a frequent contributor to the TensorFlow YouTube channel, a recognized global keynote speaker, and a prolific author.

"The book is a great introduction to understand and practice machine learning and artificial intelligence models by using TensorFlow.

—Jialin Huang PhD
Data and Applied Scientist, Microsoft

"Laurence Moroney has been a major force in building TensorFlow into one of the world's leading AI frameworks. I was privileged to support his teaching TensorFlow with deeplearning.ai and Coursera. I wish you the best in your journey learning TensorFlow. With Laurence as a teacher, great adventures await you."

—Andrew Ng
Founder, deeplearning.ai

AI

US \$65.99 CAN \$82.99
ISBN: 978-1-492-07819-7



Twitter: @oreillymedia
linkedin.com/company/oreilly-media
youtube.com/oreillymedia

Praise for *AI and Machine Learning for Coders*

“Machine learning should be in the toolbox of every great engineer in this coming decade. For people looking to get started, *AI and Machine Learning for Coders* by Laurence Moroney is the much-needed practical starting point to dive deep into deep learning, computer vision, and NLP.”

—Dominic Monn, *Machine Learning at Doist*

“The book is a great introduction to understand and practice machine learning and artificial intelligence models by using TensorFlow. It covers various deep learning models, and their practical applications, as well as how to utilize TensorFlow framework to develop and deploy ML/AI applications across platforms. I recommend it for anyone who is interested in ML and AI practice.”

—Jialin Huang PhD, *Data and Applied Scientist at Microsoft*

“Laurence’s book helped me refresh TensorFlow framework and Coursera Specialization, and motivated me to take the certification provided by Google. If you have time and you are willing to embark to an ML journey, this book is a starting point from the practice side.”

—Laura Uzcátegui, *Software Engineer*

“This book is a must-read for developers who would like to get into AI/ML. You will learn a variety of examples by coding instead of math equations.”

—Margaret Maynard-Reid, *ML Google Developer Expert*

“A practical handbook to have on your desk for implementing deep learning models.”

—Pin-Yu Chen, *Research Staff Member at IBM Research AI*

“A fun book to read and practice coding for AI and machine learning projects. Intuitive wording and graphs to explain the nonintuitive concepts and algorithms. Cool coding examples to teach you key building blocks for AI and ML. In the end, you can code AI projects for your PC program, Android, iOS and Browser!”

—*Su Fu, CEO of Alchemist*

AI and Machine Learning for Coders

A Programmer's Guide to Artificial Intelligence

Laurence Moroney

Beijing • Boston • Farnham • Sebastopol • Tokyo

O'REILLY®

AI and Machine Learning for Coders

by Laurence Moroney

Copyright © 2021 Laurence Moroney. All rights reserved.

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (<http://oreilly.com>). For more information, contact our corporate/institutional sales department: 800-998-9938 or corporate@oreilly.com.

Acquisitions Editor: Rebecca Novack

Development Editor: Angela Rufino

Production Editor: Katherine Tozer

Copyeditor: Rachel Head

Proofreader: Piper Editorial, LLC

Indexer: Judith McConville

Interior Designer: David Futato

Cover Designer: Karen Montgomery

Illustrator: O'Reilly Media, Inc.

October 2020: First Edition

Revision History for the First Edition

2020-10-01: First Release

2021-05-28: Second Release

2024-02-16: Third Release

See <http://oreilly.com/catalog/errata.csp?isbn=9781492078197> for release details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *AI and Machine Learning for Coders*, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the author, and do not represent the publisher's views. While the publisher and the author have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the author disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

978-1-492-07819-7

[LSI]

Table of Contents

Foreword.....	xiii
---------------	------

Preface.....	xv
--------------	----

Part I. Building Models

1. Introduction to TensorFlow.....	1
What Is Machine Learning?	1
Limitations of Traditional Programming	3
From Programming to Learning	5
What Is TensorFlow?	7
Using TensorFlow	8
Installing TensorFlow in Python	9
Using TensorFlow in PyCharm	9
Using TensorFlow in Google Colab	12
Getting Started with Machine Learning	13
Seeing What the Network Learned	18
Summary	19
2. Introduction to Computer Vision.....	21
Recognizing Clothing Items	21
The Data: Fashion MNIST	22
Neurons for Vision	23
Designing the Neural Network	25
The Complete Code	26
Training the Neural Network	29

Exploring the Model Output	29
Training for Longer—Discovering Overfitting	30
Stopping Training	30
Summary	32
3. Going Beyond the Basics: Detecting Features in Images.	33
Convolutions	34
Pooling	35
Implementing Convolutional Neural Networks	37
Exploring the Convolutional Network	39
Building a CNN to Distinguish Between Horses and Humans	42
The Horses or Humans Dataset	42
The Keras ImageDataGenerator	43
CNN Architecture for Horses or Humans	44
Adding Validation to the Horses or Humans Dataset	47
Testing Horse or Human Images	49
Image Augmentation	52
Transfer Learning	55
Multiclass Classification	59
Dropout Regularization	63
Summary	65
4. Using Public Datasets with TensorFlow Datasets.	67
Getting Started with TFDS	68
Using TFDS with Keras Models	70
Loading Specific Versions	73
Using Mapping Functions for Augmentation	73
Using TensorFlow Addons	74
Using Custom Splits	74
Understanding TFRecord	76
The ETL Process for Managing Data in TensorFlow	78
Optimizing the Load Phase	80
Parallelizing ETL to Improve Training Performance	81
Summary	83
5. Introduction to Natural Language Processing.	85
Encoding Language into Numbers	85
Getting Started with Tokenization	86
Turning Sentences into Sequences	87
Removing Stopwords and Cleaning Text	91
Working with Real Data Sources	93

Getting Text from TensorFlow Datasets	93
Getting Text from CSV Files	97
Getting Text from JSON Files	99
Summary	101
6. Making Sentiment Programmable Using Embeddings.	103
Establishing Meaning from Words	103
A Simple Example: Positives and Negatives	104
Going a Little Deeper: Vectors	105
Embeddings in TensorFlow	106
Building a Sarcasm Detector Using Embeddings	106
Reducing Overfitting in Language Models	109
Using the Model to Classify a Sentence	119
Visualizing the Embeddings	120
Using Pretrained Embeddings from TensorFlow Hub	123
Summary	125
7. Recurrent Neural Networks for Natural Language Processing.	127
The Basis of Recurrence	127
Extending Recurrence for Language	130
Creating a Text Classifier with RNNs	132
Stacking LSTMs	134
Using Pretrained Embeddings with RNNs	139
Summary	146
8. Using TensorFlow to Create Text.	147
Turning Sequences into Input Sequences	148
Creating the Model	152
Generating Text	154
Predicting the Next Word	154
Compounding Predictions to Generate Text	155
Extending the Dataset	156
Changing the Model Architecture	157
Improving the Data	158
Character-Based Encoding	162
Summary	163
9. Understanding Sequence and Time Series Data.	165
Common Attributes of Time Series	167
Trend	167
Seasonality	167

Autocorrelation	168
Noise	168
Techniques for Predicting Time Series	169
Naive Prediction to Create a Baseline	169
Measuring Prediction Accuracy	172
Less Naive: Using Moving Average for Prediction	172
Improving the Moving Average Analysis	173
Summary	174
10. Creating ML Models to Predict Sequences.....	175
Creating a Windowed Dataset	176
Creating a Windowed Version of the Time Series Dataset	178
Creating and Training a DNN to Fit the Sequence Data	180
Evaluating the Results of the DNN	181
Exploring the Overall Prediction	183
Tuning the Learning Rate	185
Exploring Hyperparameter Tuning with Keras Tuner	187
Summary	191
11. Using Convolutional and Recurrent Methods for Sequence Models.....	193
Convolutions for Sequence Data	193
Coding Convolutions	194
Experimenting with the Conv1D Hyperparameters	197
Using NASA Weather Data	200
Reading GISS Data in Python	201
Using RNNs for Sequence Modeling	202
Exploring a Larger Dataset	205
Using Other Recurrent Methods	207
Using Dropout	208
Using Bidirectional RNNs	211
Summary	213

Part II. Using Models

12. An Introduction to TensorFlow Lite.....	217
What Is TensorFlow Lite?	217
Walkthrough: Creating and Converting a Model to TensorFlow Lite	219
Step 1. Save the Model	220
Step 2. Convert and Save the Model	221
Step 3. Load the TFLite Model and Allocate Tensors	221

Step 4. Perform the Prediction	222
Walkthrough: Transfer Learning an Image Classifier and Converting to TensorFlow Lite	224
Step 1. Build and Save the Model	224
Step 2. Convert the Model to TensorFlow Lite	225
Step 3. Optimize the Model	227
Summary	229
13. Using TensorFlow Lite in Android Apps.....	231
What Is Android Studio?	231
Creating Your First TensorFlow Lite Android App	232
Step 1. Create a New Android Project	232
Step 2. Edit Your Layout File	234
Step 3. Add the TensorFlow Lite Dependencies	236
Step 4. Add Your TensorFlow Lite Model	238
Step 5. Write the Activity Code to Use TensorFlow Lite for Inference	238
Moving Beyond “Hello World”—Processing Images	241
TensorFlow Lite Sample Apps	244
Summary	246
14. Using TensorFlow Lite in iOS Apps.....	247
Creating Your First TensorFlow Lite App with Xcode	247
Step 1. Create a Basic iOS App	247
Step 2. Add TensorFlow Lite to Your Project	249
Step 3. Create the User Interface	250
Step 4. Add and Initialize the Model Inference Class	252
Step 5. Perform the Inference	255
Step 6. Add the Model to Your App	256
Step 7. Add the UI Logic	258
Moving Beyond “Hello World”—Processing Images	260
TensorFlow Lite Sample Apps	263
Summary	263
15. An Introduction to TensorFlow.js.....	265
What Is TensorFlow.js?	265
Installing and Using the Brackets IDE	267
Building Your First TensorFlow.js Model	268
Creating an Iris Classifier	272
Summary	276

16. Coding Techniques for Computer Vision in TensorFlow.js.....	277
JavaScript Considerations for TensorFlow Developers	278
Building a CNN in JavaScript	279
Using Callbacks for Visualization	281
Training with the MNIST Dataset	284
Running Inference on Images in TensorFlow.js	290
Summary	290
17. Reusing and Converting Python Models to JavaScript.....	293
Converting Python-Based Models to JavaScript	293
Using the Converted Models	295
Using Preconverted JavaScript Models	297
Using the Toxicity Text Classifier	297
Using MobileNet for Image Classification in the Browser	300
Using PoseNet	303
Summary	306
18. Transfer Learning in JavaScript.....	307
Transfer Learning from MobileNet	308
Step 1. Download MobileNet and Identify the Layers to Use	308
Step 2. Create Your Own Model Architecture with the Outputs from MobileNet as Its Input	310
Step 3. Gather and Format the Data	312
Step 4. Train the Model	318
Step 5. Run Inference with the Model	319
Transfer Learning from TensorFlow Hub	321
Using Models from TensorFlow.org	324
Summary	326
19. Deployment with TensorFlow Serving.....	329
What Is TensorFlow Serving?	329
Installing TensorFlow Serving	332
Installing Using Docker	332
Installing Directly on Linux	333
Building and Serving a Model	334
Exploring Server Configuration	337
Summary	340
20. AI Ethics, Fairness, and Privacy.....	341
Fairness in Programming	342
Fairness in Machine Learning	345

Tools for Fairness	347
The What-If Tool	347
Facets	348
Federated Learning	351
Step 1. Identify Available Devices for Training	351
Step 2. Identify Suitable Available Devices for Training	352
Step 3. Deploy a Trainable Model to Your Training Set	352
Step 4. Return the Results of the Training to the Server	353
Step 5. Deploy the New Master Model to the Clients	353
Secure Aggregation with Federated Learning	354
Federated Learning with TensorFlow Federated	355
Google’s AI Principles	356
Summary	357
Index.....	359

Foreword

Dear Reader,

AI is poised to transform every industry, but almost every AI application needs to be customized for its particular use. A system for reading medical records is different from one for finding defects in a factory, which is different from a product recommendation engine. For AI to reach its full potential, engineers need tools that can help them adapt the amazing capabilities available to the millions of concrete problems we wish to solve.

When I led the Google Brain team, we started to build the C++ precursor to TensorFlow called DistBelief. We were excited about the potential of harnessing thousands of CPUs to train a neural network (for instance, using 16,000 CPUs to train a cat detector on unlabeled YouTube videos). How far deep learning has come since then! What was once cutting-edge can now be done for around \$3,000 of cloud computing credits, and Google routinely trains neural networks using TPUs and GPUs at a scale that was unimaginable just years ago.

TensorFlow, too, has come a long way. It is far more usable than what we had in the early days, and has rich features ranging from modeling, to using pretrained models, to deploying on low-compute edge devices. It is today empowering hundreds of thousands of developers to build their own deep learning models.

Laurence Moroney, as Google's lead AI Advocate, has been a major force in building TensorFlow into one of the world's leading AI frameworks. I was privileged to support his teaching TensorFlow with deeplearning.ai and Coursera. These courses have reached over 80,000 learners and received numerous glowing reviews.

One unexpected aspect of friendship with Laurence is that he is also a free source of Irish poetry. He once Slacked me:

Andrew sang a sad old song
fainted through miss milliner
invitation hoops
fainted fainted
[...]

He had trained an LSTM on lyrics of traditional Irish songs and it generated these lines. If AI opens the door to fun like that, how could anyone not want to get involved? You can (i) work on exciting projects that move humanity forward, (ii) advance your career, and (iii) get free Irish poetry.

I wish you the best in your journey learning TensorFlow. With Laurence as a teacher, great adventures await you.

Keep learning,

— Andrew Ng
Founder, deeplearning.ai

Preface

Welcome to *AI and Machine Learning for Coders*, a book that I've been wanting to write for many years but that has only really become possible due to recent advances in machine learning (ML) and, in particular, TensorFlow. The goal of this book is to prepare you, as a coder, for many of the scenarios that you can address with machine learning, with the aim of equipping you to be an ML and AI developer *without* needing a PhD! I hope that you'll find it useful, and that it will empower you with the confidence to get started on this wonderful and rewarding journey.

Who Should Read This Book

If you're interested in AI and ML, and you want to get up and running quickly with building models that learn from data, this book is for you. If you're interested in getting started with common AI and ML concepts—computer vision, natural language processing, sequence modeling, and more—and want to see how neural networks can be trained to solve problems in these spaces, I think you'll enjoy this book. And if you have models that you've trained and want to get them into the hands of users on mobile, in the browser, or via the cloud, then this book is also for you!

Most of all, if you've been put off entering this valuable area of computer science because of perceived difficulty, in particular believing that you'll need to dust off your old calculus books, then fear not: this book takes a code-first approach that shows you just how easy it is to get started in the world of machine learning and artificial intelligence using Python and TensorFlow.

Why I Wrote This Book

I first got seriously involved with artificial intelligence in the spring of 1992. A freshly minted physics graduate living in London in the midst of a terrible recession, I had been unemployed for six months. The British government started a program to train 20 people in AI technology, and put out a call for applicants. I was the first

participant selected. Three months later, the program failed miserably, because while there was plenty of *theoretical* work that could be done with AI, there was no easy way to do it *practically*. One could write simple inference in a language called Prolog, and perform list processing in a language called Lisp, but there was no clear path to deploying them in industry. The famous “AI winter” followed.

Then, in 2016, while I was working at Google on a product called Firebase, the company offered machine learning training to all engineers. I sat in a room with a number of other people and listened to lectures about calculus and gradient descent. I couldn’t quite match this to a practical implementation of ML, and I was suddenly transported back to 1992. I brought feedback about this, and about how we should educate people in ML, to the TensorFlow team—and they hired me in 2017. With the release of TensorFlow 2.0 in 2018, and in particular the emphasis on high-level APIs that made it easy for developers to get started, I realized the need was there for a book that took advantage of this, and widened access to ML so that it wasn’t just for mathematicians or PhDs anymore.

I believe that more people using this technology and deploying it to end users will lead to an explosion in AI and ML that will prevent another AI winter, and change the world very much for the better. I’m already seeing the impact of this, from the work done by Google on diabetic retinopathy, through Penn State University and PlantVillage building an ML model for mobile that helps farmers diagnose cassava disease, Médecins Sans Frontières using TensorFlow models to help diagnose antibiotic resistance, and much, much more!

Navigating This Book

The book is written in two main parts. **Part I** (Chapters 1–11) talks about how to use TensorFlow to build machine learning models for a variety of scenarios. It takes you from first principles—building a model with a neural network containing only one neuron—through computer vision, natural language processing, and sequence modeling. **Part II** (Chapters 12–20) then walks you through scenarios for putting your models in people’s hands on Android and iOS, in browsers with JavaScript, and serving via the cloud. Most chapters are standalone, so you can drop in and learn something new, or, of course, you could just read the book cover to cover.

Technology You Need to Understand

The goal of the first half of the book is to help you learn how to use TensorFlow to build models with a variety of architectures. The only real prerequisite to this is understanding Python, and in particular Python notation for data and array processing. You might also want to explore Numpy, a Python library for numeric calculations. If you have no familiarity with these, they are quite easy to learn, and

you can probably pick up what you need as you go along (although some of the array notation might be a bit hard to grasp).

For the second half of the book, I generally will not teach the languages that are shown, but instead show how TensorFlow models can be used in them. So, for example, in the Android chapter ([Chapter 13](#)) you'll explore building apps in Kotlin with Android studio, and in the iOS chapter ([Chapter 14](#)) you'll explore building apps in Swift with Xcode. I won't be teaching the syntax of these languages, so if you aren't familiar with them, you may need a primer—*Learning Swift* by Jonathan Manning, Paris Buttfield-Addison, and Tim Nugent (O'Reilly) is a great choice.

Online Resources

A variety of online resources are used by, and supported in, this book. At the very least I would recommend that you keep an eye on [TensorFlow](#) and its associated [YouTube channel](#) for any updates and breaking changes to technologies discussed in the book.

The code for this book is available at <https://github.com/lmoroney/tfbook>, and I will keep it up to date there as the platform evolves.

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic

Indicates new terms, URLs, email addresses, filenames, and file extensions.

Constant width

Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, data types, environment variables, statements, and keywords.

Constant width bold

Used for emphasis in code snippets.



This element signifies a note.

Using Code Examples

This book is here to help you get your job done. In general, if example code is offered with this book, you may use it in your programs and documentation. You do not need to contact us for permission unless you're reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing examples from O'Reilly books does require permission. Answering a question by citing this book and quoting example code does not require permission. Incorporating a significant amount of example code from this book into your product's documentation does require permission.

We appreciate, but do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example: “*AI and Machine Learning for Coders*, by Laurence Moroney. Copyright 2021 Laurence Moroney, 978-1-492-07819-7.”

If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at permissions@oreilly.com.

O'Reilly Online Learning



For more than 40 years, *O'Reilly Media* has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O'Reilly's online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O'Reilly and 200+ other publishers. For more information, visit <http://oreilly.com>.

How to Contact Us

Please address comments and questions concerning this book to the publisher:

O'Reilly Media, Inc.
1005 Gravenstein Highway North
Sebastopol, CA 95472
800-998-9938 (in the United States or Canada)
707-829-0515 (international or local)
707-829-0104 (fax)

We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at <https://oreil.ly/ai-ml>.

Email bookquestions@oreilly.com to comment or ask technical questions about this book.

For news and information about our books and courses, visit <http://oreilly.com>.

Find us on Facebook: <http://facebook.com/oreilly>

Follow us on Twitter: <http://twitter.com/oreillymedia>

Watch us on YouTube: <http://www.youtube.com/oreillymedia>

Acknowledgments

I'd like to thank lots of people who have helped in the creation of this book.

Jeff Dean, who gave me the opportunity to be part of the TensorFlow team, beginning the second phase of my AI journey. There's also the rest of the team, and while there are too many to name, I'd like to call out Sarah Sirajuddin, Megan Kacholia, Martin Wicke, and Francois Chollet for their amazing leadership and engineering!

The developer relations team for TensorFlow, led by Kemal El Moujahid, Magnus Hyttsten, and Wolff Dobson, who create the platform for people to learn AI and ML with TensorFlow.

Andrew Ng, who, as well as writing the Foreword for this book, also believed in my approach to teaching TensorFlow, and with whom I created three specializations at Coursera, teaching hundreds of thousands of people how to succeed with machine learning and AI. Andrew also leads a team at deeplearning.ai who were terrific at helping me be a better machine learner, including Ortal Arel, Eddy Shu, and Ryan Keenan.

The team at O'Reilly that made this book possible: Rebecca Novack and Angela Rufino, without whose hard work I never would have gotten it done!

The amazing tech review team: Jialin Huang, Laura Uzcátegui, Lucy Wong, Margaret Maynard-Reid, Su Fu, Darren Richardson, Dominic Monn, and Pin-Yu.

And of course, most important of all (even more than Jeff and Andrew ;)) is my family, who make the most important stuff meaningful: my wife Rebecca Moroney, my daughter Claudia Moroney, and my son Christopher Moroney. Thanks to you all for making life more amazing than I ever thought it could be.

PART I

Building Models

Introduction to TensorFlow

When it comes to creating artificial intelligence (AI), machine learning (ML) and deep learning are a great place to begin. When getting started, however, it's easy to get overwhelmed by the options and all the new terminology. This book aims to demystify things for programmers, taking you through writing code to implement concepts of machine learning and deep learning; and building models that behave more as a human does, with scenarios like computer vision, natural language processing (NLP), and more. Thus, they become a form of synthesized, or artificial, intelligence.

But when we refer to *machine learning*, what in fact is this phenomenon? Let's take a quick look at that, and consider it from a programmer's perspective before we go any further. After that, this chapter will show you how to install the tools of the trade, from TensorFlow itself to environments where you can code and debug your TensorFlow models.

What Is Machine Learning?

Before we get into the ins and outs of ML, let's consider how it evolved from traditional programming. We'll start by examining what traditional programming is, then consider cases where it is limited. Then we'll see how ML evolved to handle those cases, and as a result has opened up new opportunities to implement new scenarios, unlocking many of the concepts of artificial intelligence.

Traditional programming involves us writing rules, expressed in a programming language, that act on data and give us answers. This applies just about everywhere that something can be programmed with code.

For example, consider a game like the popular Breakout. Code determines the movement of the ball, the score, and the various conditions for winning or losing the game. Think about the scenario where the ball bounces off a brick, like in [Figure 1-1](#).

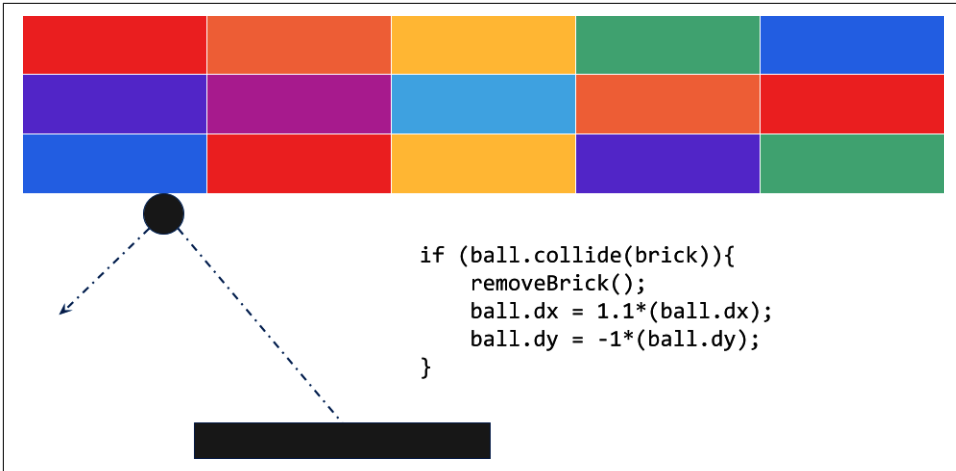


Figure 1-1. Code in a Breakout game

Here, the motion of the ball can be determined by its `dx` and `dy` properties. When it hits a brick, the brick is removed, and the velocity of the ball increases and changes direction. The code acts on data about the game situation.

Alternatively, consider a financial services scenario. You have data about a company's stock, such as its current price and current earnings. You can calculate a valuable ratio called the P/E (for price divided by earnings) with code like that in Figure 1-2.

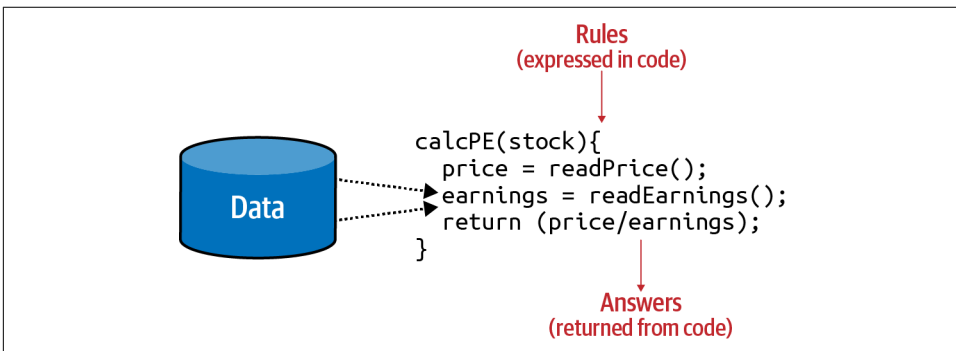


Figure 1-2. Code in a financial services scenario

Your code reads the price, reads the earnings, and returns a value that is the former divided by the latter.

If I were to try to sum up traditional programming like this into a single diagram, it might look like **Figure 1-3**.



Figure 1-3. High-level view of traditional programming

As you can see, you have rules expressed in a programming language. These rules act on data, and the result is answers.

Limitations of Traditional Programming

The model from **Figure 1-3** has been the backbone of development since its inception. But it has an inherent limitation: namely, that only scenarios that can be implemented are ones for which you can derive rules. What about other scenarios? Usually, they are infeasible to develop because the code is too complex. It's just not possible to write code to handle them.

Consider, for example, activity detection. Fitness monitors that can detect our activity are a recent innovation, not just because of the availability of cheap and small hardware, but also because the algorithms to handle detection weren't previously feasible. Let's explore why.

Figure 1-4 shows a naive activity detection algorithm for walking. It can consider the person's speed. If it's less than a particular value, we can determine that they are probably walking.

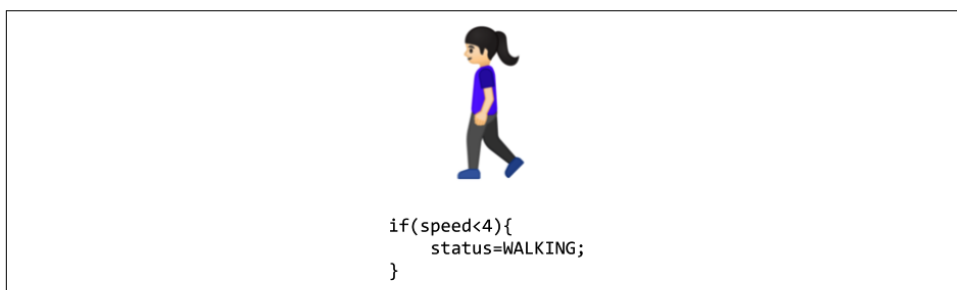


Figure 1-4. Algorithm for activity detection

Given that our data is speed, we could extend this to detect if they are running (Figure 1-5).

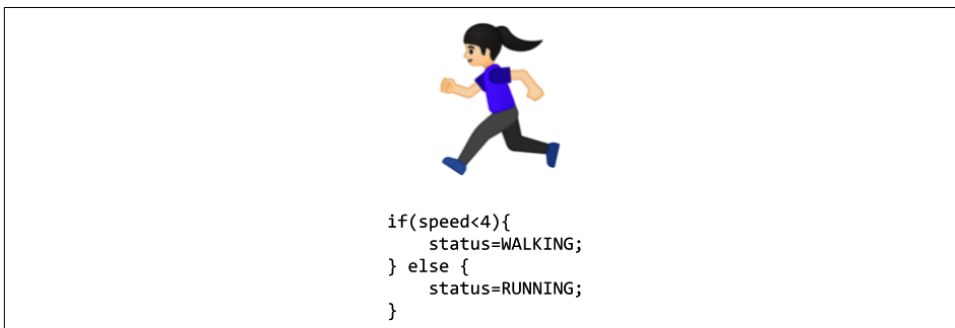


Figure 1-5. Extending the algorithm for running

As you can see, going by the speed, we might say if it is less than a particular value (say, 4 mph) the person is walking, and otherwise they are running. It still sort of works.

Now suppose we want to extend this to another popular fitness activity, biking. The algorithm could look like Figure 1-6.

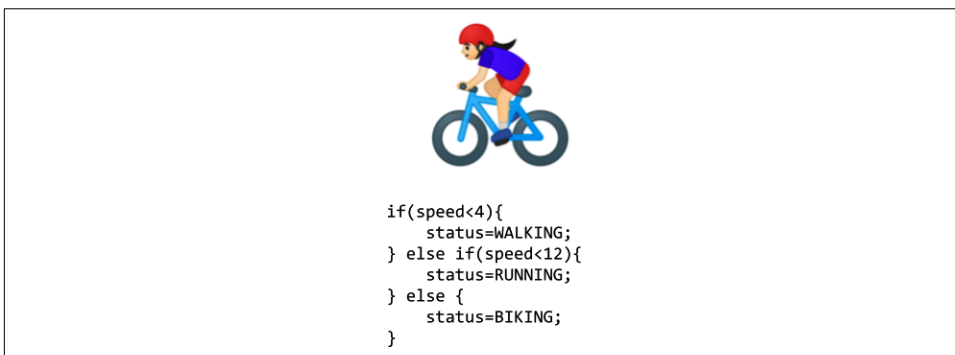


Figure 1-6. Extending the algorithm for biking

I know it's naive in that it just detects speed—some people run faster than others, and you might run downhill faster than you cycle uphill, for example. But on the whole, it still works. However, what happens if we want to implement another scenario, such as golfing (Figure 1-7)?

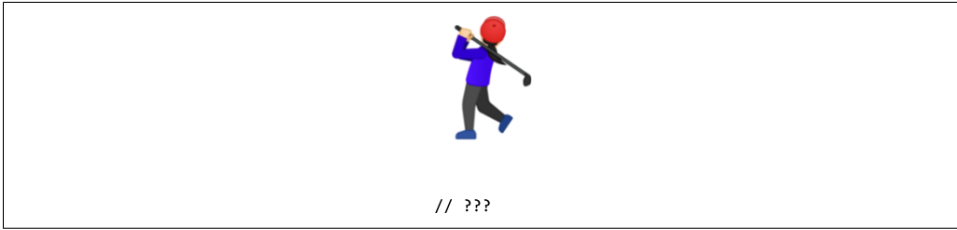


Figure 1-7. How do we write a golfing algorithm?

We're now stuck. How do we determine that someone is golfing using this methodology? The person might walk for a bit, stop, do some activity, walk for a bit more, stop, etc. But how can we tell this is golf?

Our ability to detect this activity using traditional rules has hit a wall. But maybe there's a better way.

Enter machine learning.

From Programming to Learning

Let's look back at the diagram that we used to demonstrate what traditional programming is (Figure 1-8). Here we have rules that act on data and give us answers. In our activity detection scenario, the data was the speed at which the person was moving; from that we could write rules to detect their activity, be it walking, biking, or running. We hit a wall when it came to golfing, because we couldn't come up with rules to determine what that activity looks like.



Figure 1-8. The traditional programming flow

But what would happen if we were to flip the axes around on this diagram? Instead of us coming up with the *rules*, what if we were to come up with the *answers*, and along with the data have a way of figuring out what the rules might be?

Figure 1-9 shows what this would look like. We can consider this high-level diagram to define *machine learning*.



Figure 1-9. Changing the axes to get machine learning

So what are the implications of this? Well, now instead of *us* trying to figure out what the rules are, we get lots of data about our scenario, we label that data, and the computer can figure out what the rules are that make one piece of data match a particular label and another piece of data match a different label.

How would this work for our activity detection scenario? Well, we can look at all the sensors that give us data about this person. If they have a wearable that detects information such as heart rate, location, speed, etc.—and if we collect a lot of instances of this data while they’re doing different activities—we end up with a scenario of having data that says “This is what walking looks like,” “This is what running looks like,” and so on (Figure 1-10).

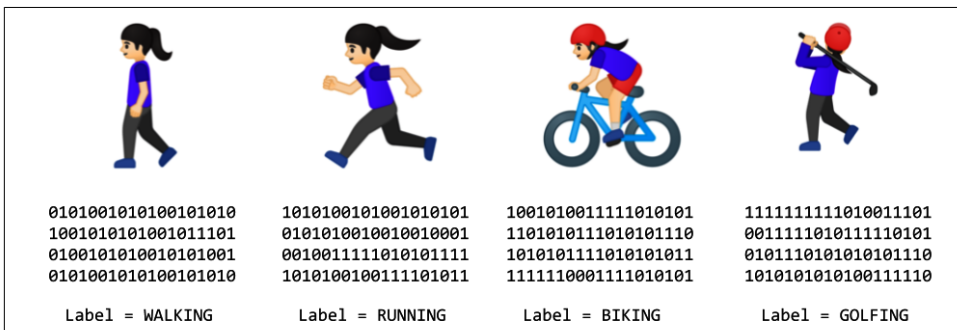


Figure 1-10. From coding to ML: gathering and labeling data

Now our job as programmers changes from figuring out the rules, to determining the activities, to writing the code that matches the data to the labels. If we can do this, then we can expand the scenarios that we can implement with code. Machine learning is a technique that enables us to do this, but in order to get started, we’ll need a framework—and that’s where TensorFlow enters the picture. In the next section we’ll take a look at what it is and how to install it, and then later in this chapter you’ll write your first code that learns the pattern between two values, like in the preceding scenario. It’s a simple “Hello World” scenario, but it has the same foundational code pattern that’s used in extremely complex ones.

The field of artificial intelligence is large and abstract, encompassing everything to do with making computers think and act the way human beings do. One of the ways a human takes on new behaviors is through learning by example. The discipline of

machine learning can thus be thought of as an on-ramp to the development of artificial intelligence. Through it, a machine can learn to see like a human (a field called *computer vision*), read text like a human (natural language processing), and much more. We'll be covering the basics of machine learning in this book, using the TensorFlow framework.

What Is TensorFlow?

TensorFlow is an open source platform for creating and using machine learning models. It implements many of the common algorithms and patterns needed for machine learning, saving you from needing to learn all the underlying math and logic and enabling you to just to focus on your scenario. It's aimed at everyone from hobbyists, to professional developers, to researchers pushing the boundaries of artificial intelligence. Importantly, it also supports deployment of models to the web, cloud, mobile, and embedded systems. We'll be covering each of these scenarios in this book.

The high-level architecture of TensorFlow can be seen in [Figure 1-11](#).

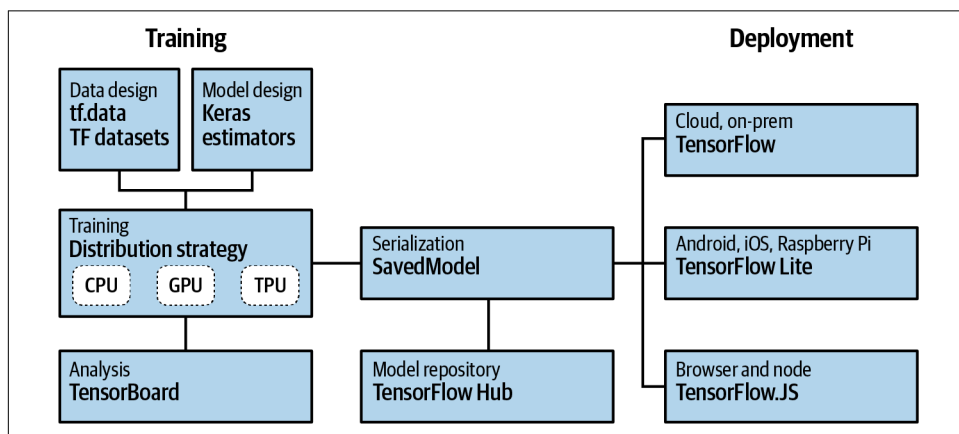


Figure 1-11. TensorFlow high-level architecture

The process of creating machine learning models is called *training*. This is where a computer uses a set of algorithms to learn about inputs and what distinguishes them from each other. So, for example, if you want a computer to recognize cats and dogs, you can use lots of pictures of both to create a model, and the computer will use that model to try to figure out what makes a cat a cat, and what makes a dog a dog. Once the model is trained, the process of having it recognize or categorize future inputs is called *inference*.

So, for training models, there are several things that you need to support. First is a set of APIs for designing the models themselves. With TensorFlow there are three main ways to do this: you can code everything by hand, where you figure out the logic for

how a computer can learn and then implement that in code (not recommended); you can use built-in *estimators*, which are already-implemented neural networks that you can customize; or you can use Keras, a high-level API that allows you to encapsulate common machine learning paradigms in code. This book will primarily focus on using the Keras APIs when creating models.

There are many ways to train a model. For the most part, you'll probably just use a single chip, be it a central processing unit (CPU), a graphics processing unit (GPU), or something new called a *tensor processing unit* (TPU). In more advanced working and research environments, parallel training across multiple chips can be used, employing a *distribution strategy* where training spans multiple chips. TensorFlow supports this too.

The lifeblood of any model is its data. As discussed earlier, if you want to create a model that can recognize cats and dogs, it needs to be trained with lots of examples of cats and dogs. But how can you manage these examples? You'll see, over time, that this can often involve a lot more coding than the creation of the models themselves. TensorFlow ships with APIs to try to ease this process, called TensorFlow Data Services. For learning, they include lots of preprocessed datasets that you can use with a single line of code. They also give you the tools for processing raw data to make it easier to use.

Beyond creating models, you'll also need to be able to get them into people's hands where they can be used. To this end, TensorFlow includes APIs for *serving*, where you can provide model inference over an HTTP connection for cloud or web users. For models to run on mobile or embedded systems, there's TensorFlow Lite, which provides tools for model inference on Android and iOS as well as Linux-based embedded systems such as a Raspberry Pi. A fork of TensorFlow Lite, called TensorFlow Lite Micro (TFLM), also provides inference on microcontrollers through an emerging concept known as TinyML. Finally, if you want to provide models to your browser or Node.js users, TensorFlow.js offers the ability to train and execute models in this way.

Next, I'll show you how to install TensorFlow so that you can get started creating and using ML models with it.

Using TensorFlow

In this section, we'll look at the three main ways that you can install and use TensorFlow. We'll start with how to install it on your developer box using the command line. We'll then explore using the popular PyCharm IDE (integrated development environment) to install TensorFlow. Finally, we'll look at Google Colab and how it can be used to access TensorFlow code with a cloud-based backend in your browser.

Installing TensorFlow in Python

TensorFlow supports the creation of models using multiple languages, including Python, Swift, Java, and more. In this book we'll focus on using Python, which is the de facto language for machine learning due to its extensive support for mathematical models. If you don't have it already, I strongly recommend you visit [Python](#) to get up and running with it, and [learnpython.org](#) to learn the Python language syntax.

With Python there are many ways to install frameworks, but the default one supported by the TensorFlow team is pip.

So, in your Python environment, installing TensorFlow is as easy as using:

```
pip install tensorflow
```

Note that starting with version 2.1, this will install the GPU version of TensorFlow by default. Prior to that, it used the CPU version. So, before installing, make sure you have a supported GPU and all the requisite drivers for it. Details on this are available at [TensorFlow](#).

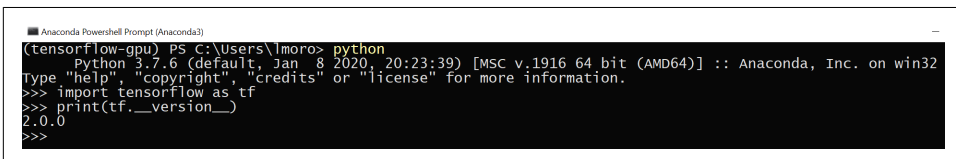
If you don't have the required GPU or drivers, you can still install the CPU version of TensorFlow on any Linux, PC, or Mac with:

```
pip install tensorflow-cpu
```

Once you're up and running, you can test your TensorFlow version with the following code:

```
import tensorflow as tf
print(tf.__version__)
```

You should see output like that in [Figure 1-12](#). This will print the currently running version of TensorFlow—here you can see that version 2.0.0 is installed.

A screenshot of an Anaconda PowerShell Prompt window. The title bar reads "Anaconda PowerShell Prompt (Anaconda3)". The command prompt shows the user running 'python', which opens a Python 3.7.6 shell. The user then enters 'import tensorflow as tf' and 'print(tf.__version__)', resulting in the output '2.0.0'.

```
■ Anaconda PowerShell Prompt (Anaconda3)
(tensorflow-gpu) PS C:\Users\lmoro> python
Python 3.7.6 (default, Jan 8 2020, 20:23:39) [MSC v.1916 64 bit (AMD64)] :: Anaconda, Inc. on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> import tensorflow as tf
>>> print(tf.__version__)
2.0.0
>>>
```

Figure 1-12. Running TensorFlow in Python

Using TensorFlow in PyCharm

I'm particularly fond of using the [free community version of PyCharm](#) for building models using TensorFlow. PyCharm is useful for many reasons, but one of my favorites is that it makes the management of virtual environments easy. This means you can have Python environments with versions of tools such as TensorFlow that are specific to your particular project. So, for example, if you want to use TensorFlow 2.0 in one project and TensorFlow 2.1 in another, you can separate these with virtual

environments and not have to deal with installing/uninstalling dependencies when you switch. Additionally, with PyCharm you can do step-by-step debugging of your Python code—a must, especially if you’re just getting started.

For example, in [Figure 1-13](#) I have a new project that is called *example1*, and I’m specifying that I am going to create a new environment using Conda. When I create the project I’ll have a clean new virtual Python environment into which I can install any version of TensorFlow I want.

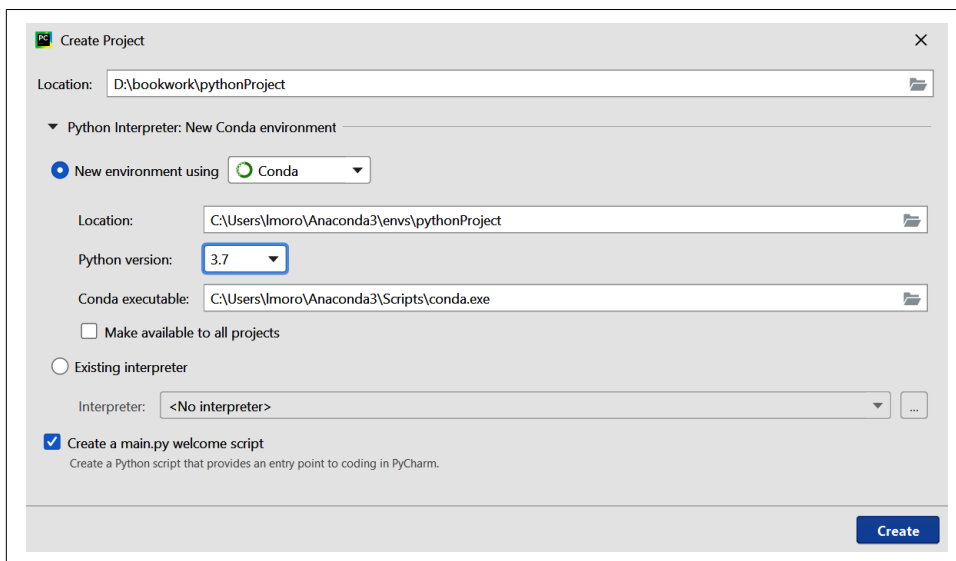


Figure 1-13. Creating a new virtual environment using PyCharm

Once you’ve created a project, you can open the File → Settings dialog and choose the entry for “Project: *<your project name>*” from the menu on the left. You’ll then see choices to change the settings for the Project Interpreter and the Project Structure. Choose the Project Interpreter link, and you’ll see the interpreter that you’re using, as well as a list of packages that are installed in this virtual environment ([Figure 1-14](#)).

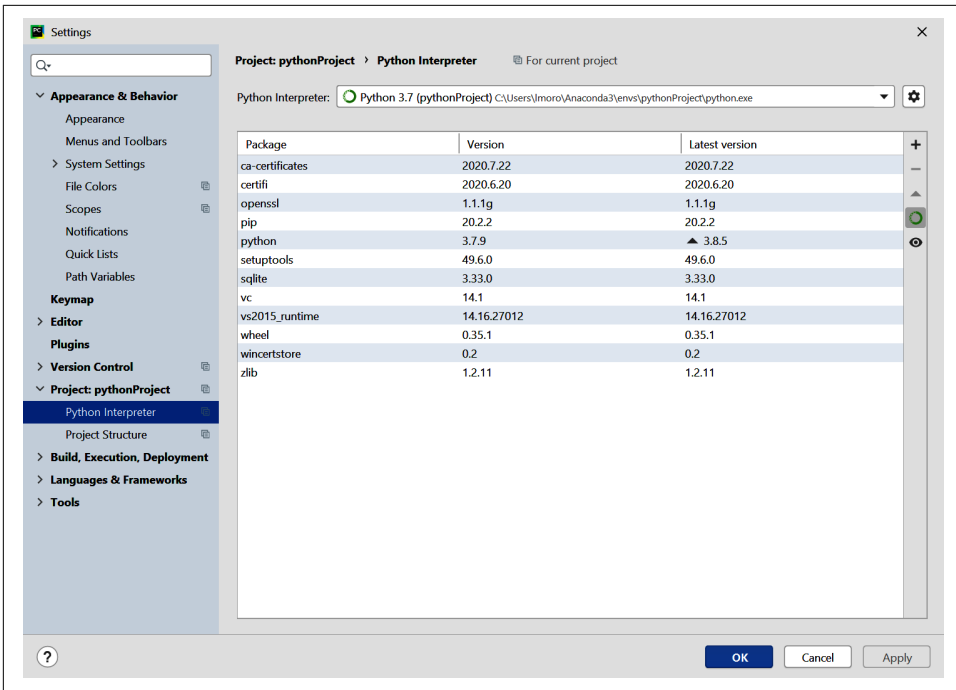


Figure 1-14. Adding packages to a virtual environment

Click the + button on the right, and a dialog will open showing the packages that are currently available. Type “tensorflow” into the search box and you’ll see all available packages with “tensorflow” in the name (Figure 1-15).

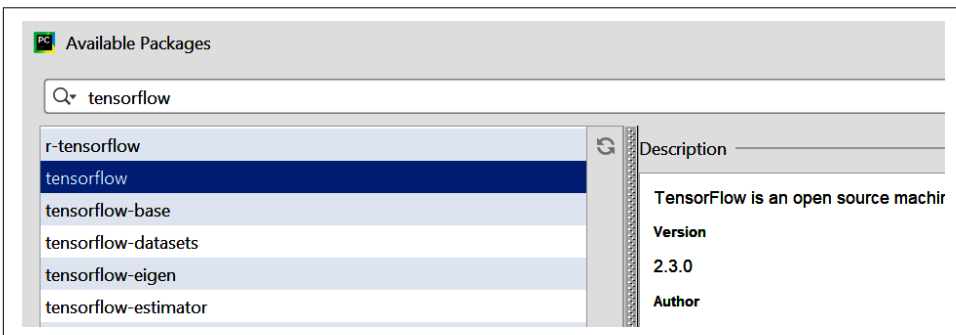


Figure 1-15. Installing TensorFlow with PyCharm

Once you’ve selected TensorFlow, or any other package you want to install, and then click the Install Package button, PyCharm will do the rest.

Once TensorFlow is installed, you can now write and debug your TensorFlow code in Python.

Using TensorFlow in Google Colab

Another option, which is perhaps easiest for getting started, is to use [Google Colab](#), a hosted Python environment that you can access via a browser. What's really neat about Colab is that it provides GPU and TPU backends so you can train models using state-of-the-art hardware at no cost.

When you visit the Colab website, you'll be given the option to open previous Colabs or start a new notebook, as shown in [Figure 1-16](#).

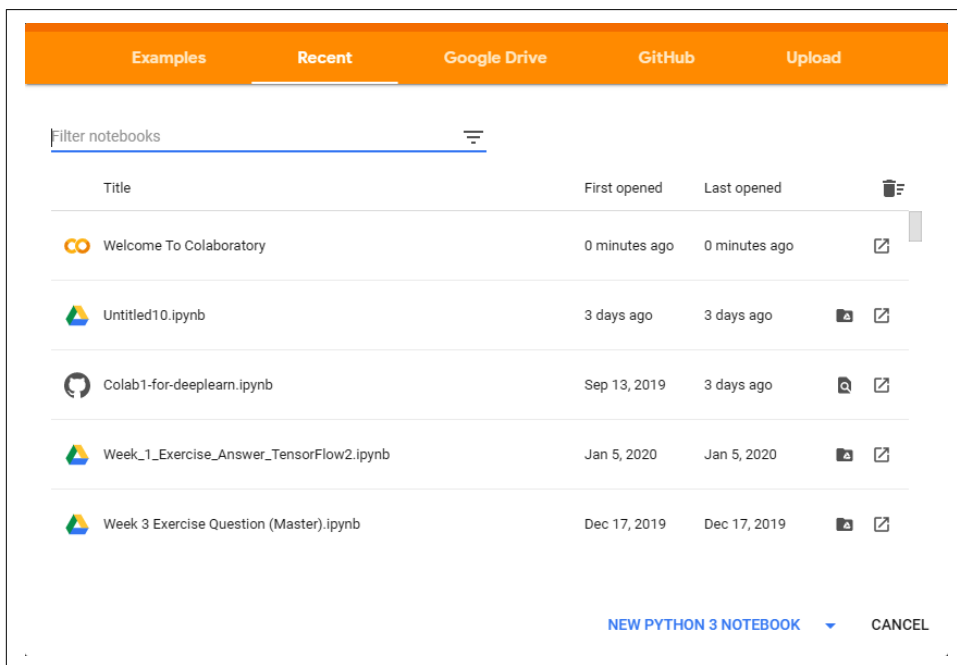


Figure 1-16. Getting started with Google Colab

Clicking the New Python 3 Notebook link will open the editor, where you can add panes of code or text ([Figure 1-17](#)). You can execute the code by clicking the Play button (the arrow) to the left of the pane.

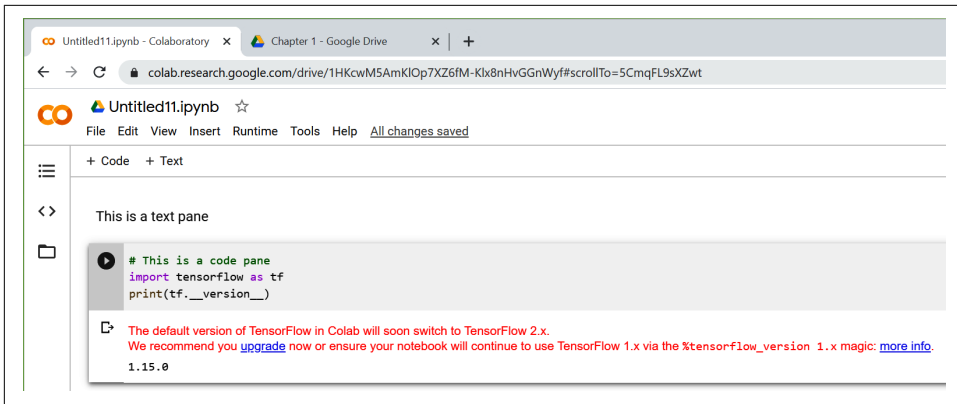


Figure 1-17. Running TensorFlow code in Colab

It's always a good idea to check the TensorFlow version, as shown here, to be sure you're running the correct version. Often Colab's built-in TensorFlow will be a version or two behind the latest release. If that's the case you can update it with `pip install` as shown earlier, by simply using a block of code like this:

```
!pip install tensorflow==2.1.0
```

Once you run this command, your current environment within Colab will use the desired version of TensorFlow.

Getting Started with Machine Learning

As we saw earlier in the chapter, the machine learning paradigm is one where you have data, that data is labeled, and you want to figure out the rules that match the data to the labels. The simplest possible scenario to show this in code is as follows. Consider these two sets of numbers:

```
X = -1, 0, 1, 2, 3, 4
Y = -3, -1, 1, 3, 5, 7
```

There's a relationship between the X and Y values (for example, if X is -1 then Y is -3, if X is 3 then Y is 5, and so on). Can you see it?

After a few seconds you probably saw that the pattern here is $Y = 2X - 1$. How did you get that? Different people work it out in different ways, but I typically hear the observation that X increases by 1 in its sequence, and Y increases by 2; thus, $Y = 2X$ +/- something. They then look at when $X = 0$ and see that $Y = -1$, so they figure that the answer could be $Y = 2X - 1$. Next they look at the other values and see that this hypothesis "fits," and the answer is $Y = 2X - 1$.

That's very similar to the machine learning process. Let's take a look at some TensorFlow code that you could write to have a neural network do this figuring out for you.

Here's the full code, using the TensorFlow Keras APIs. Don't worry if it doesn't make sense yet; we'll go through it line by line:

```
import tensorflow as tf
import numpy as np
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense

model = Sequential([Dense(units=1, input_shape=[1])])
model.compile(optimizer='sgd', loss='mean_squared_error')

xs = np.array([-1.0, 0.0, 1.0, 2.0, 3.0, 4.0], dtype=float)
ys = np.array([-3.0, -1.0, 1.0, 3.0, 5.0, 7.0], dtype=float)

model.fit(xs, ys, epochs=500)

print(model.predict([10.0]))
```

Let's start with the first line. You've probably heard of neural networks, and you've probably seen diagrams that explain them using layers of interconnected neurons, a little like [Figure 1-18](#).

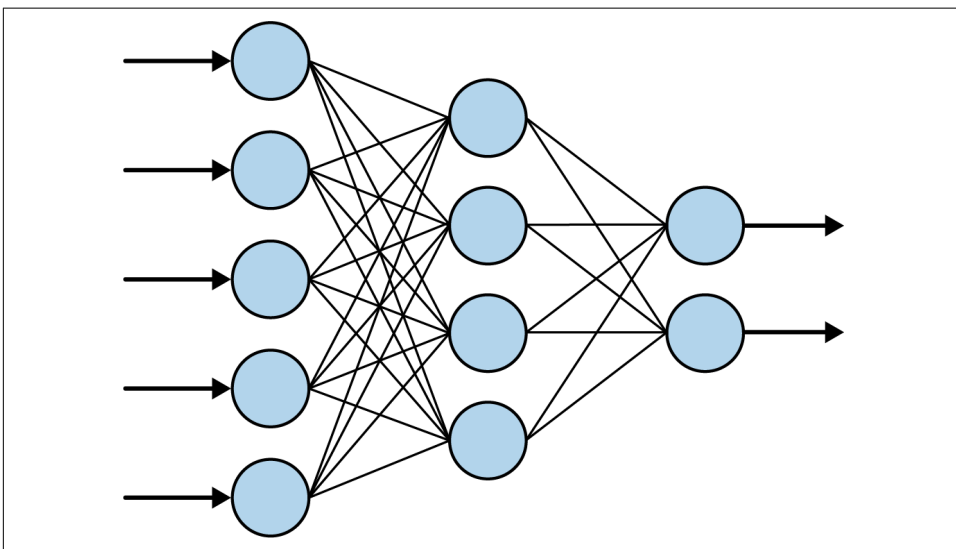


Figure 1-18. A typical neural network

When you see a neural network like this, consider each of the “circles” to be a *neuron*, and each of the columns of circles to be a *layer*. So, in [Figure 1-18](#), there are three layers: the first has five neurons, the second has four, and the third has two.

If we look back at our code and look at just the first line, we'll see that we're defining the simplest possible neural network. There's only one layer, and it contains only one neuron:

```
model = Sequential([Dense(units=1, input_shape=[1])])
```

When using TensorFlow, you define your layers using `Sequential`. Inside the `Sequential`, you then specify what each layer looks like. We only have one line inside our `Sequential`, so we have only one layer.

You then define what the layer looks like using the `keras.layers` API. There are lots of different layer types, but here we're using a `Dense` layer. "Dense" means a set of fully (or densely) connected neurons, which is what you can see in [Figure 1-18](#) where every neuron is connected to every neuron in the next layer. It's the most common form of layer type. Our `Dense` layer has `units=1` specified, so we have just one dense layer with one neuron in our entire neural network. Finally, when you specify the *first* layer in a neural network (in this case, it's our only layer), you have to tell it what the shape of the input data is. In this case our input data is our `X`, which is just a single value, so we specify that that's its shape.

The next line is where the fun really begins. Let's look at it again:

```
model.compile(optimizer='sgd', loss='mean_squared_error')
```

If you've done anything with machine learning before, you've probably seen that it involves a lot of mathematics. If you haven't done calculus in years it might have seemed like a barrier to entry. Here's the part where the math comes in—it's the core to machine learning.

In a scenario such as this one, the computer has *no idea* what the relationship between `X` and `Y` is. So it will make a guess. Say for example it guesses that $Y = 10X + 10$. It then needs to measure how good or how bad that guess is. That's the job of the *loss function*.

It already knows the answers when `X` is `-1`, `0`, `1`, `2`, `3`, and `4`, so the loss function can compare these to the answers for the guessed relationship. If it guessed $Y = 10X + 10$, then when `X` is `-1`, `Y` will be `0`. The correct answer there was `-3`, so it's a bit off. But when `X` is `4`, the guessed answer is `50`, whereas the correct one is `7`. That's really far off.

Armed with this knowledge, the computer can then make another guess. That's the job of the *optimizer*. This is where the heavy calculus is used, but with TensorFlow, that can be hidden from you. You just pick the appropriate optimizer to use for different scenarios. In this case we picked one called `sgd`, which stands for *stochastic gradient descent*—a complex mathematical function that, when given the values, the previous guess, and the results of calculating the errors (or loss) on that guess, can

then generate another one. Over time, its job is to minimize the loss, and by so doing bring the guessed formula closer and closer to the correct answer.

Next, we simply format our numbers into the data format that the layers expect. In Python, there's a library called Numpy that TensorFlow can use, and here we put our numbers into a Numpy array to make it easy to process them:

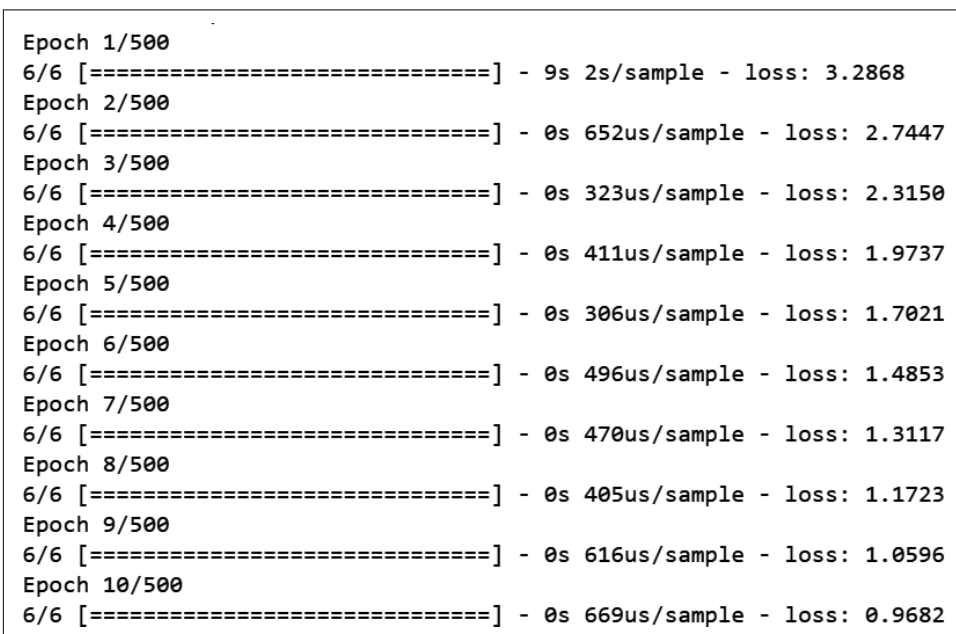
```
xs = np.array([-1.0, 0.0, 1.0, 2.0, 3.0, 4.0], dtype=float)
ys = np.array([-3.0, -1.0, 1.0, 3.0, 5.0, 7.0], dtype=float)
```

The *learning* process will then begin with the `model.fit` command, like this:

```
model.fit(xs, ys, epochs=500)
```

You can read this as “fit the Xs to the Ys, and try it 500 times.” So, on the first try, the computer will guess the relationship (i.e., something like $Y = 10X + 10$), and measure how good or bad that guess was. It will then feed those results to the optimizer, which will generate another guess. This process will then be repeated, with the logic being that the loss (or error) will go down over time, and as a result the “guess” will get better and better.

Figure 1-19 shows a screenshot of this running in a Colab notebook. Take a look at the loss values over time.



```
Epoch 1/500
6/6 [=====] - 9s 2s/sample - loss: 3.2868
Epoch 2/500
6/6 [=====] - 0s 652us/sample - loss: 2.7447
Epoch 3/500
6/6 [=====] - 0s 323us/sample - loss: 2.3150
Epoch 4/500
6/6 [=====] - 0s 411us/sample - loss: 1.9737
Epoch 5/500
6/6 [=====] - 0s 306us/sample - loss: 1.7021
Epoch 6/500
6/6 [=====] - 0s 496us/sample - loss: 1.4853
Epoch 7/500
6/6 [=====] - 0s 470us/sample - loss: 1.3117
Epoch 8/500
6/6 [=====] - 0s 405us/sample - loss: 1.1723
Epoch 9/500
6/6 [=====] - 0s 616us/sample - loss: 1.0596
Epoch 10/500
6/6 [=====] - 0s 669us/sample - loss: 0.9682
```

Figure 1-19. Training the neural network